Function
(C Programming Language)

Dr. Thien Huynh-The

Dept. Comp. Commun. Eng.

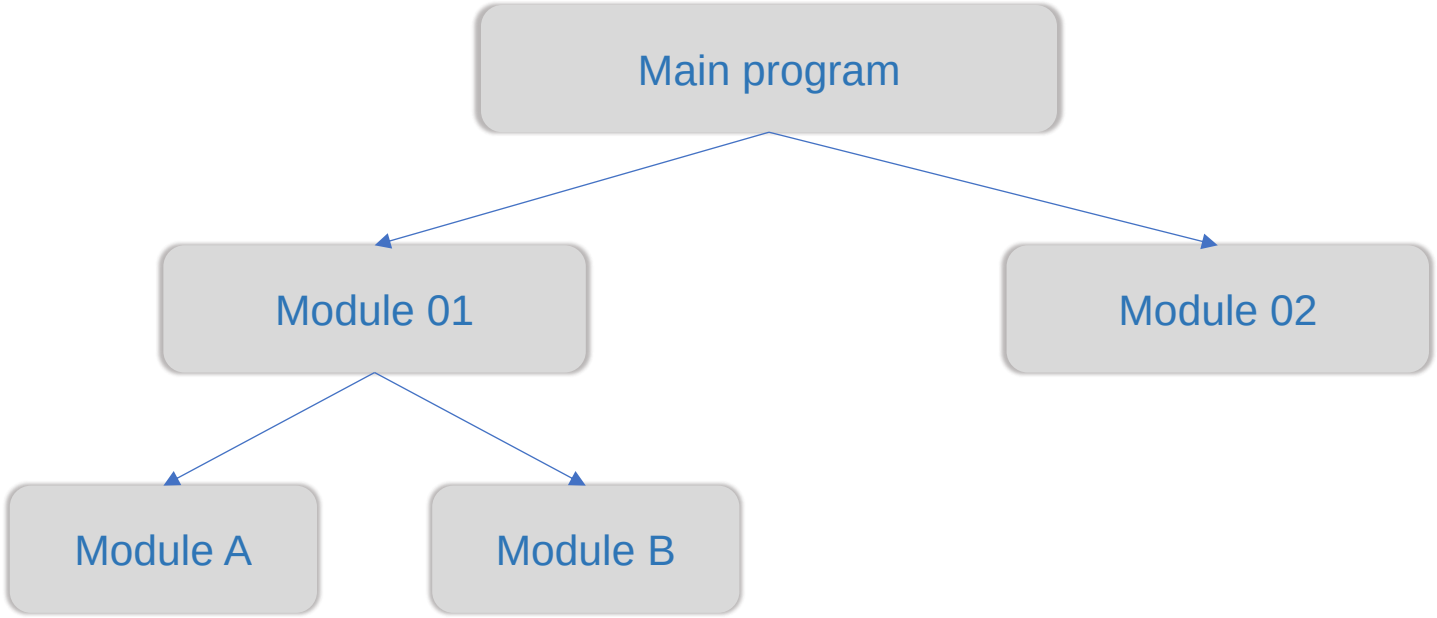
HCMC Technology and Education

Content

- Introduction
- Program Modules in C
- Math Library Functions
- Functions
- Function Definitions
- Function Prototypes
- Function Call Stack and Activation Records
- Headers
- Calling Functions: Call-by-Value and Call-by- Reference
- Random Number Generation
- Example: A Game of Chance
- Storage Classes
- Scope Rules
- Recursion
- Example Using Recursion: Fibonacci Series
- Recursion vs. Iteration

Introduction

- Divide and conquer
  - Construct a program from smaller pieces or components
    - These smaller pieces are called modules
  - Each piece more manageable than the original program



Program Modules in C

- Functions
  - Modules in C
  - Programs combine user-defined functions with library functions
    - C standard library has a wide variety of functions
- Function calls
  - Invoking functions
    - Provide function name and arguments (data)
    - Function performs operations or manipulations
    - Function returns results
  - Function call analogy:
    - Boss asks worker to complete task
      - Worker gets information, does task, returns result
      - Information hiding: boss does not know details

Program Modules in C

- Familiarize yourself with the rich collection of functions in the C Standard Library.
- Avoid reinventing the wheel. When possible, use C Standard Library functions instead of writing new functions. This can reduce program development time.
- Using the functions in the C Standard Library helps make programs more portable.

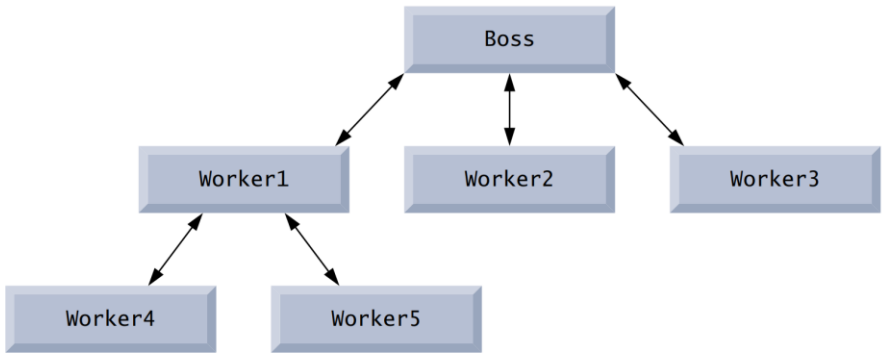


Fig. 5.1 | Hierarchical boss-function/worker-function relationship.

Math Library Functions

- Math library functions
  - perform common mathematical calculations

```
#include <math.h>
```

- Format for calling functions
  - FunctionName(argument);
    - If multiple arguments, use comma-separated list

```
printf( "%.2f", sqrt( 900.0 ) );
```

    - Calls function sqrt, which returns the square root of its argument
    - All math functions return data type double
  - Arguments may be constants, variables, or expressions
- Include the math header by using the preprocessor directive `#include <math.h>` when using functions in the math library.

C Math Library Functions

Function	Description	Example
`sqrt(x)`	square root of x	`sqrt(900.0)` is `30.0` `sqrt(9.0)` is `3.0`
`cbrt(x)`	cube root of x (C99 and C11 only)	`cbrt(27.0)` is `3.0` `cbrt(-8.0)` is `-2.0`
`exp(x)`	exponential function e^x	`exp(1.0)` is `2.718282` `exp(2.0)` is `7.389056`
`log(x)`	natural logarithm of x (base e)	`log(2.718282)` is `1.0` `log(7.389056)` is `2.0`

`log10(x)`	logarithm of x (base 10)	`log10(1.0)` is `0.0`
		`log10(10.0)` is `1.0`
		`log10(100.0)` is `2.0`

`fabs(x)`	absolute value of x as a floating-point number	`fabs(13.5)` is `13.5`
		`fabs(0.0)` is `0.0`
		`fabs(-13.5)` is `13.5`

`ceil(x)`	rounds x to the smallest integer not less than x	`ceil(9.2)` is `10.0` `ceil(-9.8)` is `-9.0`
`floor(x)`	rounds x to the largest integer not greater than x	`floor(9.2)` is `9.0` `floor(-9.8)` is `-10.0`
`pow(x, y)`	x raised to power y (x^y)	`pow(2, 7)` is `128.0` `pow(9, .5)` is `3.0`

Fig. 5.2 | Commonly used math library functions. (Part 1 of 2.)

C Math Library Functions

Function	Description	Example
`fmod(x, y)`	remainder of x/y as a floating-point number	`fmod(13.657, 2.333)` is `1.992`
`sin(x)`	trigonometric sine of x (x in radians)	`sin(0.0)` is `0.0`
`cos(x)`	trigonometric cosine of x (x in radians)	`cos(0.0)` is `1.0`
`tan(x)`	trigonometric tangent of x (x in radians)	`tan(0.0)` is `0.0`

Fig. 5.2 | Commonly used math library functions. (Part 2 of 2.)

Functions

- Functions
  - Modularize a program
  - All variables defined inside functions are local variables
    - Known only in function defined
  - Parameters
    - Communicate information between functions
    - Local variables
- Benefits of functions
  - Divide and conquer
    - Manageable program development
  - Software reusability
    - Use existing functions as building blocks for new programs
    - Abstraction - hide internal details (library functions)
  - Avoid code repetition

Functions

- In programs containing many functions, main is often implemented as a group of calls to functions.
- Each function should be limited to performing a single, well-defined task, and the function name should effectively express that task.
- If you cannot choose a concise name that expresses what the function does, it is possible that your function is attempting to perform too many diverse tasks.

Function Definitions

- Function definition format

```
return-value-type  function-name( parameter-list )
   {
      declarations and statements
   }
```

  - Function-name: any valid identifier
  - Return-value-type: data type of the result (default `int`)
    - `void` – indicates that the function returns nothing
  - Parameter-list: comma separated list, declares parameters
    - A type must be listed explicitly for each parameter unless, the parameter is of type `int`

Function Definitions

- Function definition format (continued)

```
return-value-type  function-name( parameter-list )
   {
      declarations and statements
   }
```

  - Definitions and statements: function body (block)
    - Variables can be defined inside blocks (can be nested)
    - Functions can not be defined inside other functions
  - Returning control
    - If nothing returned

      `return;`

      or, until reaches right brace
    - If something returned

      `return expression;`

Examples

```c
// Fig. 5.3: fig05_03.c
// Creating and using a programmer-defined function.
#include <stdio.h>

int square(int y); // function prototype

int main(void)
{
   // loop 10 times and calculate and output square of x each time
   for (int x = 1; x <= 10; ++x) {
      printf("%d  ", square(x)); // function call
   }

   puts("");
}

// square function definition returns the square of its parameter
int square(int y) // y is a copy of the argument to the function
{
   return y * y; // returns the square of y as an int
}
```

Function prototype indicates function will be defined later in the program

Call to square function

Function definition

```
1  4  9  16  25  36  49  64  81  100
```

Functions

- Place a blank line between function definitions
- Defining a function inside another function is a syntax error
- Omitting the return-value-type in a function definition is a syntax error
- Forgetting to return a value from a function that is supposed to return a value can lead to unexpected errors
- Returning a value from a function with a void return type is a syntax error.
- Even though an omitted return type defaults to int, always state the return type explicitly.

  `double x, double y` is different with `double x, y`
- Consider dividing the function into smaller functions

Examples

```c
// Fig. 5.4: fig05_04.c
// Finding the maximum of three integers.
#include <stdio.h>

int maximum(int x, int y, int z); // function prototype

int main(void)
{
   int number1; // first integer entered by the user
   int number2; // second integer entered by the user
   int number3; // third integer entered by the user

   printf("%s", "Enter three integers: ");
   scanf("%d%d%d", &number1, &number2, &number3);

   // number1, number2 and number3 are arguments
   // to the maximum function call
   printf("Maximum is: %d\n", maximum(number1, number2, number3));
}

// Function maximum definition
// x, y and z are parameters
int maximum(int x, int y, int z)
{
   int max = x; // assume x is largest

   if (y > max) { // if y is larger than max,
      max = y; // assign y to max
   }

   if (z > max) { // if z is larger than max,
      max = z; // assign z to max
   }

   return max; // max is largest value
}
```

Function prototype

Function call

Function definition

Function Prototypes

- Function prototype
  - Function name
  - Parameters – what the function takes in
  - Return type – data type function returns (default `int`)
  - Used to validate functions
  - Prototype only needed if function definition comes after use in program
  - The function with the prototype

```
int maximum( int x, int y, int z );
```

    - Takes in 3 `ints`
    - Returns an `int`

- Promotion rules and conversions
  - Converting to lower types can lead to errors

Function

- Use `#include` preprocessor directives to obtain function prototypes for the standard library functions
- Parameter names are sometimes included in function prototypes (our preference) for documentation purposes.
- Forgetting the semicolon at the end of a function prototype is a syntax error.

Data Types

Data type	`printf` conversion specification	`scanf` conversion specification
Floating-point types		
`long double`	`%Lf`	`%Lf`
`double`	`%f`	`%lf`
`float`	`%f`	`%f`

Fig. 5.5 | Arithmetic data types and their conversion specifications. (Part 1 of 2.)

Data Types

Data type	printf conversion specification	scanf conversion specification
Integer types		
unsigned long long int	%llu	%llu
long long int	%lld	%lld
unsigned long int	%lu	%lu
long int	%ld	%ld
unsigned int	%u	%u
int	%d	%d
unsigned short	%hu	%hu
short	%hd	%hd
char	%c	%c

Fig. 5.5 | Arithmetic data types and their conversion specifications. (Part 2 of 2.)

Data Types

- Converting from a higher data type in the promotion hierarchy to a lower type can change the data value.
- Forgetting a function prototype causes a syntax error.
- A function prototype placed outside any function definition applies to all calls to the function appearing after the function prototype in the file.
- A function prototype placed in a function applies only to calls made in that function.

Function Call Stack and Activation Records

- Program execution stack
  - A stack is a last-in, first-out (LIFO) data structure
    - Anything put into the stack is placed “on top”
    - The only data that can be taken out is the data on top
  - C uses a program execution stack to keep track of which functions have been called
    - When a function is called, it is placed on top of the stack
    - When a function ends, it is taken off the stack and control returns to the function immediately below it
  - Calling more functions than C can handle at once is known as a “stack overflow error”

Headers

- Header files
  - Contain function prototypes for library functions

    `<stdlib.h>`, `<math.h>`, etc
  - Load with #include <filename>

    ```
    #include <math.h>
    ```
- Custom header files
  - Create file with functions
  - Save as filename.h
  - Load in other files with `#include "filename.h"`
  - Reuse functions

Headers

Header	Explanation
`<assert.h>`	Contains information for adding diagnostics that aid program debugging.
`<ctype.h>`	Contains function prototypes for functions that test characters for certain properties, and function prototypes for functions that can be used to convert lowercase letters to uppercase letters and vice versa.
`<errno.h>`	Defines macros that are useful for reporting error conditions.
`<float.h>`	Contains the floating-point size limits of the system.
`<limits.h>`	Contains the integral size limits of the system.
`<locale.h>`	Contains function prototypes and other information that enables a program to be modified for the current locale on which it's running. The notion of locale enables the computer system to handle different conventions for expressing data such as dates, times, currency amounts and large numbers throughout the world.

Fig. 5.10 | Some of the standard library headers. (Part 1 of 2.)

Headers

Header	Explanation
`<math.h>`	Contains function prototypes for math library functions.
`<setjmp.h>`	Contains function prototypes for functions that allow bypassing of the usual function call and return sequence.
`<signal.h>`	Contains function prototypes and macros to handle various conditions that may arise during program execution.
`<stdarg.h>`	Defines macros for dealing with a list of arguments to a function whose number and types are unknown.
`<stddef.h>`	Contains common type definitions used by C for performing calculations.
`<stdio.h>`	Contains function prototypes for the standard input/output library functions, and information used by them.
`<stdlib.h>`	Contains function prototypes for conversions of numbers to text and text to numbers, memory allocation, random numbers and other utility functions.
`<string.h>`	Contains function prototypes for string-processing functions.
`<time.h>`	Contains function prototypes and types for manipulating the time and date.

Fig. 5.10 | Some of the standard library headers. (Part 2 of 2.)

Calling Functions

- Call by value
  - Copy of argument passed to function
  - Changes in function do not effect original
  - Use when function does not need to modify argument
    - Avoids accidental changes
- Call by reference
  - Passes original argument
  - Changes in function effect original
  - Only used with trusted functions
- For now, we focus on call by value

Random Number Generation

- `rand` function
  - Load `<stdlib.h>`
  - Returns "random" number between 0 and RAND_MAX (at least 32767)

```
i = rand();
```

  - Pseudorandom
    - Preset sequence of "random" numbers
    - Same sequence for every function call
- Scaling
  - To get a random number between 1 and n

```
1 + ( rand() % n )
```

    - `rand() %n` returns a number between 0 and n - 1
    - Add 1 to make random number between 1 and n

```
1 + ( rand() % 6)
```

      - number between 1 and 6

Random Number Generation

```c
// Fig. 5.11: fig05_11.c
// Shifted, scaled random integers produced by 1 + rand() % 6.
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
   // loop 20 times
   for (unsigned int i = 1; i <= 20; ++i) {

      // pick random number from 1 to 6 and output it
      printf("%10d", 1 + (rand() % 6));

      // if counter is divisible by 5, begin new line of output
      if (i % 5 == 0) {
         puts("");
      }
   }
}
```

Generates a random number between 1 and 6

```
         6         6         5         5         6
         5         1         1         5         3
         6         6         2         4         2
         6         2         3         4         1
```

Fig. 5.11 | Shifted, scaled random integers produced by `1 + rand() % 6`. (Part 2 of 2.)

Examples

```c
// Fig. 5.12: fig05_12.c
// Rolling a six-sided die 60,000,000 times.
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
   unsigned int frequency1 = 0; // rolled 1 counter
   unsigned int frequency2 = 0; // rolled 2 counter
   unsigned int frequency3 = 0; // rolled 3 counter
   unsigned int frequency4 = 0; // rolled 4 counter
   unsigned int frequency5 = 0; // rolled 5 counter
   unsigned int frequency6 = 0; // rolled 6 counter

   // loop 60000000 times and summarize results
   for (unsigned int roll = 1; roll <= 60000000; ++roll) {
      int face = 1 + rand() % 6; // random number from 1 to 6

      // determine face value and increment appropriate counter
      switch (face) {

         case 1: // rolled 1
            ++frequency1;
            break;

         case 2: // rolled 2
            ++frequency2;
            break;
```

Fig. 5.12 | Rolling a six-sided die 60,000,000 times. (Part 1 of 2.)

Examples

```
29
30          case 3: // rolled 3
31             ++frequency3;
32             break;
33
34          case 4: // rolled 4
35             ++frequency4;
36             break;
37
38          case 5: // rolled 5
39             ++frequency5;
40             break;
41
42          case 6: // rolled 6
43             ++frequency6;
44             break; // optional
45       }
46    }
47
48    // display results in tabular format
49    printf("%s%13s\n", "Face", "Frequency");
50    printf("   1%13u\n", frequency1);
51    printf("   2%13u\n", frequency2);
52    printf("   3%13u\n", frequency3);
53    printf("   4%13u\n", frequency4);
54    printf("   5%13u\n", frequency5);
55    printf("   6%13u\n", frequency6);
56 }
```

Random Number Generation

- `srand` function

  `<stdlib.h>`

  - Takes an integer seed and jumps to that location in its "random" sequence

    ```
    srand( seed );
    ```

  - `srand( time( NULL ) );/*load <time.h> */`
    - `time( NULL )`
    - Returns the number of seconds that have passed since January 1, 1970
    - “Randomizes" the seed

Examples

```c
// Fig. 5.13: fig05_13.c
// Randomizing the die-rolling program.
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
   unsigned int seed; // number used to seed the random number generator

   printf("%s", "Enter seed: ");
   scanf("%u", &seed); // note %u for unsigned int

   srand(seed); // seed the random number generator

   // loop 10 times
   for (unsigned int i = 1; i <= 10; ++i) {

      // pick a random number from 1 to 6 and output it
      printf("%10d", 1 + (rand() % 6));

      // if counter is divisible by 5, begin a new line of output
      if (i % 5 == 0) {
         puts("");
      }
   }
}
```

Seeds the rand function

```
Enter seed: 67
         6         1         4         6         2
         1         6         1         6         4
```

```
Enter seed: 867
         2         4         6         1         6
         1         1         3         6         2
```

Fig. 5.13 | Randomizing the die-rolling program. (Part 1 of 2.)

Error: Using `srand` in place of `rand` to generate random numbers.

Examples: A Game of Chance

- Craps simulator
- Rules
  - Roll two dices
    - 7 or 11 on first throw, player wins
    - 2, 3, or 12 on first throw, player loses
    - 4, 5, 6, 8, 9, 10 - value becomes player's "point"
  - Player must roll his point before rolling 7 to win

Examples: A Game of Chance

Examples: A Game of Chance

```c
// Fig. 5.14: fig05_14.c
// Simulating the game of craps.
#include <stdio.h>
#include <stdlib.h>
#include <time.h> // contains prototype for function time

// enumeration constants represent game status
enum Status { CONTINUE, WON, LOST };

int rollDice(void); // function prototype

int main(void)
{
   // randomize random number generator using current time
   srand(time(NULL));

   int myPoint; // player must make this point to win
   enum Status gameStatus; // can contain CONTINUE, WON, or LOST
   int sum = rollDice(); // first roll of the dice

   // determine game status based on sum of dice
   switch(sum) {

      // win on first roll
      case 7: // 7 is a winner
      case 11: // 11 is a winner
         gameStatus = WON;
         break;

      // lose on first roll
      case 2: // 2 is a loser
      case 3: // 3 is a loser
      case 12: // 12 is a loser
         gameStatus = LOST;
         break;

      // remember point
      default:
         gameStatus = CONTINUE; // player should keep rolling
         myPoint = sum; // remember the point
         printf("Point is %d\n", myPoint);
         break; // optional
   }

```

enum (enumeration) assigns numerical values to CONTINUE, WON, and LOST

Fig. 5.14 | Simulating the game of craps. (Part 1 of 2.)

Examples: A Game of Chance

```c
    // while game not complete
    while (CONTINUE == gameStatus) { // player should keep rolling
       sum = rollDice(); // roll dice again

       // determine game status
       if (sum == myPoint) { // win by making point
          gameStatus = WON;
       }
       else {
          if (7 == sum) { // lose by rolling 7
             gameStatus = LOST;
          }
       }
    }

    // display won or lost message
    if (WON == gameStatus) { // did player win?
       puts("Player wins");
    }
    else { // player lost
       puts("Player loses");
    }
 }

 // roll dice, calculate sum and display results
 int rollDice(void)
 {
    int die1 = 1 + (rand() % 6); // pick random die1 value
    int die2 = 1 + (rand() % 6); // pick random die2 value

    // display results of this roll
    printf("Player rolled %d + %d = %d\n", die1, die2, die1 + die2);
    return die1 + die2; // return sum of dice
 }
```

Fig. 5.14 | Simulating the game of craps. (Part 2 of 2.)

Examples: A Game of Chance

Player wins on the first roll

```
Player rolled 5 + 6 = 11
Player wins
```

Player wins on a subsequent roll

```
Player rolled 4 + 1 = 5
Point is 5
Player rolled 6 + 2 = 8
Player rolled 2 + 1 = 3
Player rolled 3 + 2 = 5
Player wins
```

Player loses on the first roll

```
Player rolled 1 + 1 = 2
Player loses
```

Player loses on a subsequent roll

```
Player rolled 6 + 4 = 10
Point is 10
Player rolled 3 + 4 = 7
Player loses
```

Storage Classes

- Storage class specifiers
  - Storage duration – how long an object exists in memory
  - Scope – where object can be referenced in program
  - Linkage – specifies the files in which an identifier is known
- Automatic storage
  - Object created and destroyed within its block
  - `auto`: default for local variables

    ```
    auto double x, y;
    ```

  - `register`: tries to put variable into high-speed registers
  - Can only be used for automatic variables

    ```
    register int counter = 1;
    ```

Storage Classes

- Static storage
  - Variables exist for entire program execution
  - Default value of zero
  - `static`: local variables defined in functions.
    - Keep value after function ends
    - Only known in their own function
  - `extern`: default for global variables and functions
    - Known in any function

Scope Rules

- File scope
  - Identifier defined outside function, known in all functions
  - Used for global variables, function definitions, function prototypes
- Function scope
  - Can only be referenced inside a function body
  - Used only for labels (`start:`, `case:` , etc.)

Scope Rules

- Block scope
  - Identifier declared inside a block
    - Block scope begins at definition, ends at right brace
  - Used for variables, function parameters (local variables of function)
  - Outer blocks "hidden" from inner blocks if there is a variable with the same name in the inner block
- Function prototype scope
  - Used for identifiers in parameter list

Scope Rules

```c
// Fig. 5.16: fig05_16.c
// Scoping.
#include <stdio.h>

void useLocal(void); // function prototype
void useStaticLocal(void); // function prototype
void useGlobal(void); // function prototype

int x = 1; // global variable

int main(void)
{
   int x = 5; // local variable to main

   printf("local x in outer scope of main is %d\n", x);

   { // start new scope
      int x = 7; // local variable to new scope

      printf("local x in inner scope of main is %d\n", x);
   } // end new scope

   printf("local x in outer scope of main is %d\n", x);

   useLocal(); // useLocal has automatic local x
   useStaticLocal(); // useStaticLocal has static local x
   useGlobal(); // useGlobal uses global x
   useLocal(); // useLocal reinitializes automatic local x
   useStaticLocal(); // static local x retains its prior value
   useGlobal(); // global x also retains its value

   printf("\nlocal x in main is %d\n", x);
}

// useLocal reinitializes local variable x during each call
void useLocal(void)
{
   int x = 25; // initialized each time useLocal is called

   printf("\nlocal x in useLocal is %d after entering useLocal\n", x);
   ++x;
   printf("local x in useLocal is %d before exiting useLocal\n", x);
}
```

Global variable with file scope

Variable with block scope

Variable with block scope

Variable with block scope

Scope Rules

```c
// useStaticLocal initializes static local variable x only the first time
// the function is called; value of x is saved between calls to this
// function
void useStaticLocal(void)
{
   // initialized once
   static int x = 50;

   printf("\nlocal static x is %d on entering useStaticLocal\n", x);
   ++x;
   printf("local static x is %d on exiting useStaticLocal\n", x);
}

// function useGlobal modifies global variable x during each call
void useGlobal(void)
{
   printf("\nglobal x is %d on entering useGlobal\n", x);
   x *= 10;
   printf("global x is %d on exiting useGlobal\n", x);
}
```

Static variable with block scope

Global variable

```
local x in outer scope of main is 5
local x in inner scope of main is 7
local x in outer scope of main is 5

local x in useLocal is 25 after entering useLocal
local x in useLocal is 26 before exiting useLocal

local static x is 50 on entering useStaticLocal
local static x is 51 on exiting useStaticLocal

global x is 1 on entering useGlobal
global x is 10 on exiting useGlobal

local x in useLocal is 25 after entering useLocal
local x in useLocal is 26 before exiting useLocal

local static x is 51 on entering useStaticLocal
local static x is 52 on exiting useStaticLocal

global x is 10 on entering useGlobal
global x is 100 on exiting useGlobal

local x in main is 5
```

Recursion

- Recursive functions
  - Functions that call themselves
  - Can only solve a base case
  - Divide a problem up into
    - What it can do
    - What it cannot do
      - What it cannot do resembles original problem
      - The function launches a new copy of itself (recursion step) to solve what it cannot do
  - Eventually base case gets solved
    - Gets plugged in, works its way up and solves whole problem

Recursion

- Example: factorials
  - 5! = 5 * 4 * 3 * 2 * 1
  - Notice that
    - 5! = 5 * 4!
    - 4! = 4 * 3! ...
  - Can compute factorials recursively
  - Solve base case (1! = 0! = 1) then plug in
    - 2! = 2 * 1! = 2 * 1 = 2;
    - 3! = 3 * 2! = 3 * 2 = 6;

Recursion

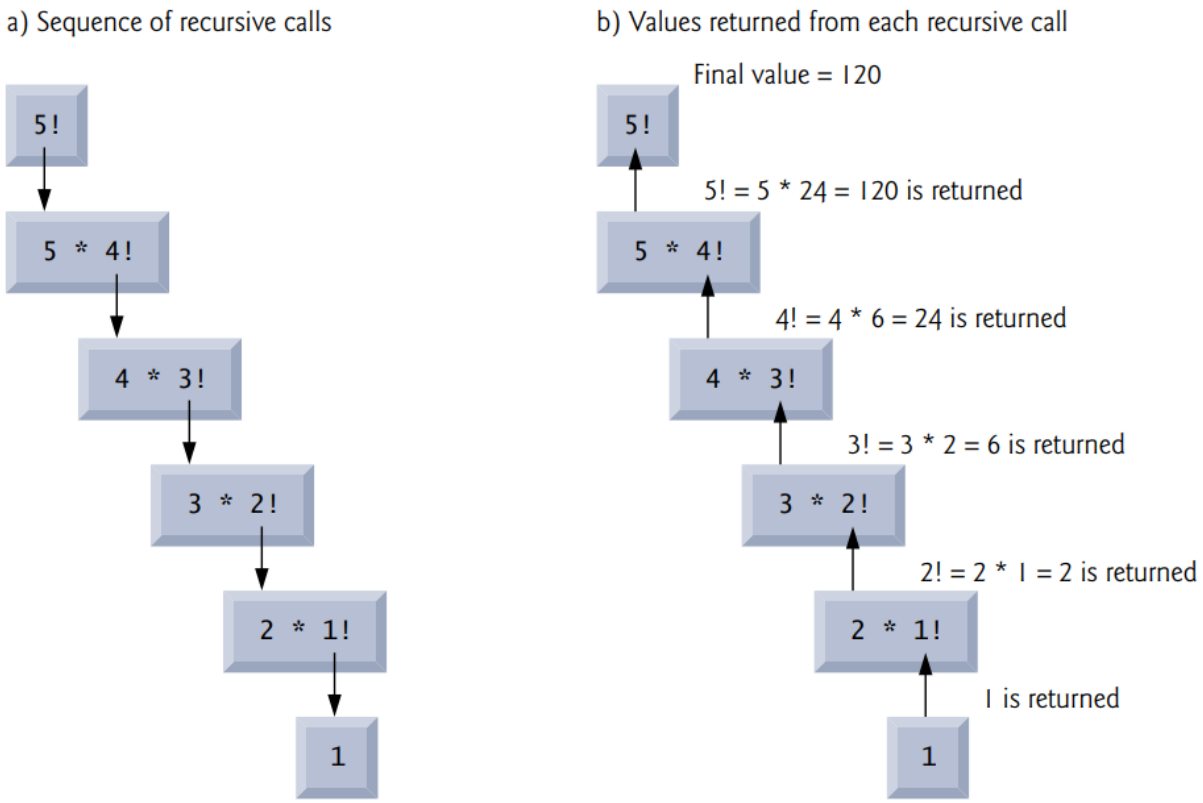


Fig. 5.17 | Recursive evaluation of 5!.

Recursion

```c
// Fig. 5.18: fig05_18.c
// Recursive factorial function.
#include <stdio.h>

unsigned long long int factorial(unsigned int number);

int main(void)
{
   // during each iteration, calculate
   // factorial(i) and display result
   for (unsigned int i = 0; i <= 21; ++i) {
      printf("%u! = %llu\n", i, factorial(i));
   }
}

// recursive definition of function factorial
unsigned long long int factorial(unsigned int number)
{
   // base case
   if (number <= 1) {
      return 1;
   }
   else { // recursive step
      return (number * factorial(number - 1));
   }
}
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
11! = 39916800
12! = 479001600
13! = 6227020800
14! = 87178291200
15! = 1307674368000
16! = 20922789888000
17! = 355687428096000
18! = 6402373705728000
19! = 121645100408832000
20! = 2432902008176640000
21! = 14197454024290336768
```

Recursion

- Forgetting to return a value from a recursive function when one is needed.
- Either omitting the base case, or writing the recursion step incorrectly so that it does not converge on the base case, will cause infinite recursion, eventually exhausting memory.

Examples: Fibonacci Series

- Fibonacci series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, …
  - Each number is the sum of the previous two
  - Can be solved recursively:

```
fib( n ) = fib( n - 1 ) + fib( n – 2 )
```

- Code for the fibonacci function

```
long fibonacci( long n )
{
  if (n == 0 || n == 1)  // base case
    return n;
  else
    return fibonacci( n - 1) +
     fibonacci( n – 2 );
}
```

Examples: Fibonacci Series

```c
// Fig. 5.19: fig05_19.c
// Recursive fibonacci function
#include <stdio.h>

unsigned long long int fibonacci(unsigned int n); // function prototype

int main(void)
{
   unsigned int number; // number input by user

   // obtain integer from user
   printf("%s", "Enter an integer: ");
   scanf("%u", &number);

   // calculate fibonacci value for number input by user
   unsigned long long int result = fibonacci(number);

   // display result
   printf("Fibonacci(%u) = %llu\n", number, result);
}

// Recursive definition of function fibonacci
unsigned long long int fibonacci(unsigned int n)
{
   // base case
   if (0 == n || 1 == n) {
      return n;
   }
   else { // recursive step
      return fibonacci(n - 1) + fibonacci(n - 2);
   }
}
```

Examples: Fibonacci Series

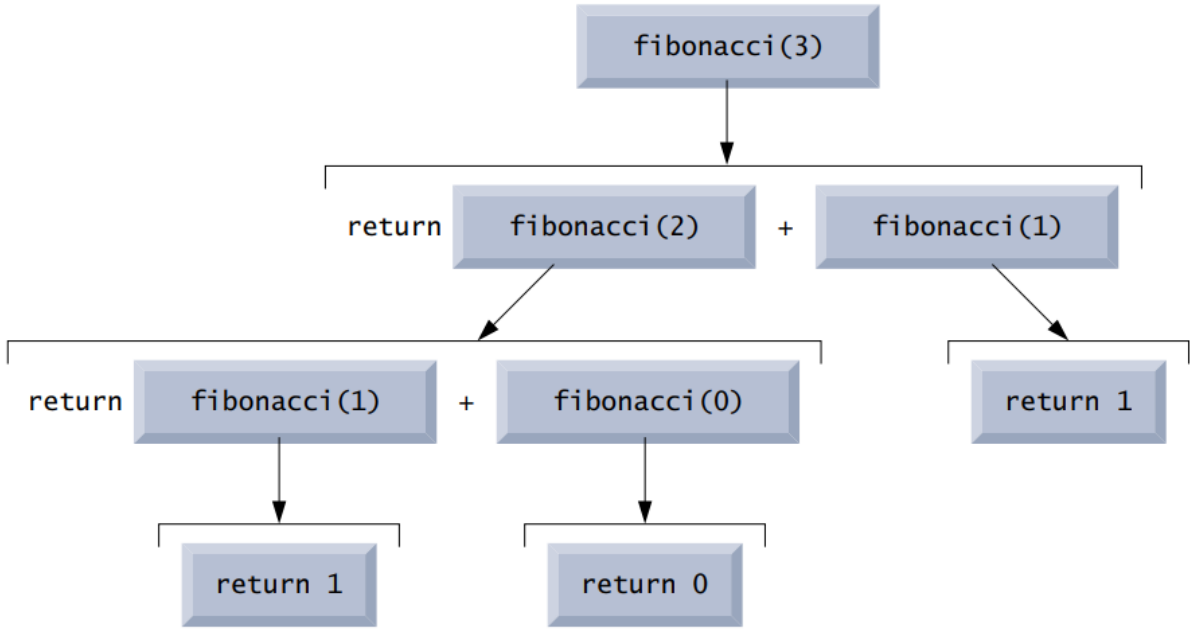


Fig. 5.20 | Set of recursive calls for `fibonacci(3)`.

Recursion

- Writing programs that depend on the order of evaluation of the operands of operators other than `&&`, `||`, `?:`, and the comma (`,`) operator can lead to errors
- Programs that depend on the order of evaluation of the operands of operators other than `&&`, `||`, `?:`, and the comma (`,`) operator can function differently on systems with different compilers.
- Avoid Fibonacci-style recursive programs which result in an exponential “explosion” of calls.

Recursion vs Iteration

- Repetition
  - Iteration: explicit loop
  - Recursion: repeated function calls
- Termination
  - Iteration: loop condition fails
  - Recursion: base case recognized
- Both can have infinite loops
- Balance
  - Choice between performance (iteration) and good software engineering (recursion

Recursion

- Any problem that can be solved recursively can also be solved iteratively (nonrecursively).
- Avoid using recursion in performance situations. Recursive calls take time and consume additional memory.
- Accidentally having a nonrecursive function call itself either directly, or indirectly through another function.
- A heavily functionalized program—as compared to a monolithic (i.e., one-piece) program without functions—makes potentially large numbers of function calls, and these consume execution time on a computer’s processor(s).

Recursion

Recursion examples and exercises	
Chapter 5	*Chapter 7*
Factorial function	Maze traversal
Fibonacci function	*Chapter 8*
Greatest common divisor	Printing a string input at the keyboard backward
Multiply two integers	*Chapter 12*
Raising an integer to an integer power	Search a linked list
Towers of Hanoi	Print a linked list backward
Recursive `main`	Binary tree insert
Visualizing recursion	Preorder traversal of a binary tree
Chapter 6	Inorder traversal of a binary tree
Sum the elements of an array	Postorder traversal of a binary tree
Print an array	Printing trees
Print an array backward	*Appendix D*
Print a string backward	Selection sort
Check whether a string is a palindrome	Quicksort
Minimum value in an array	*Appendix E*
Linear search	Fibonacci function
Binary search	
Eight Queens	

Fig. 5.21 | Recursion examples and exercises in the text.

Exercises

- Viết chtrình trong đó có hàm tự định nghĩa xác định số nguyên tố
  - Nhập vào 1 số nguyên dương. Kiểm tra số đó có phải có nguyên tố hay ko?
  - Tiếp tục nhập vào khoảng giá trị [a,b]. Tìm và in ra tất cả các số nguyên tố có trong khoảng đó
- Viết chtrình sử dụng hàm tự định nghĩa
  - Nhập vào 1 số nguyên dương. Kiểm tra số đó có phải là số amstrong

    Ví dụ:
    - $4 = 4^1$
    - $371 = 3^3+7^3+1^3$
    - $1634=1^4+6^4+3^4+4^4$
  - Nhập vào 1 số nguyên dương. Kiểm tra số đó có phải là số hoàn hảo

    Ví dụ: 6=1+2+3 (6 = tổng các ước số của nó – ngoại trừ chính nó)

    28=1+2+4+7+14